

HealthPro — Моє Здоров'я

Завдання для Replit Agent: Підготовка до Фази 2

Версія: 1.0 | Дата: 25 квітня 2026 р. | Після: Фаза 1 (100% виконано)

КОНТЕКСТ ТА МЕТА

Фазу 1 рефакторингу завершено повністю: 28 тестів проходять, модульна архітектура побудована, core/platform.js готовий. Перед переходом до Фази 2 (Capacitor Android) та Етапу 5 (SQLite) необхідно закрити два технічних завдання, які були виявлені під час аналізу звіту: розширення тестів для критичної функції isPillDueToday та налаштування CI/CD через GitHub Actions.



ЗАДАЧА А — Тести для isPillDueToday

Функція isPillDueToday визначає, чи потрібно приймати ліки сьогодні. Робота з датами та днями тижня — найчастіше джерело прихованих багів у медичних застосунках. Помилка тут = пропущений прийом ліків або хибне нагадування. Це найвищий пріоритет перед релізом.

А.1 Що тестувати

Сценарій	Умова	Очікуваний результат
Щодня	days: 'daily', будь-який день	true
Конкретна дата — сьогодні	days: 'date', date = today()	true
Конкретна дата — вчора	days: 'date', date = yesterday	false
Конкретна дата — завтра	days: 'date', date = tomorrow	false
По днях тижня — збіг	days: 'mon', поточний день — понеділок	true
По днях тижня — не збіг	days: 'mon', поточний день — вівторок	false

Кілька днів — збіг	days: 'mon,wed', день — середа	true
Кілька днів — не збіг	days: 'mon,wed', день — четвер	false
Порожні days	days: '', date: null	false (не крашити)
Невалідні days	days: 'xyz', будь-який день	false (не крашити)
days: null	pill без поля days	false (не крашити)

А.2 Файл та розміщення

Створити файл tests/pill-schedule.test.js.

tests/setup.js вже існує з mock для localStorage — імпортувати його.

Структура файлу:

```
import { isPillDueToday } from '../src/features/meds/schedule.js'
import { vi } from 'vitest'
// Мокаємо today() з core/utils.js для контролю поточної дати
vi.mock('../src/core/utils.js', () => ({ today: () => '2026-04-25' })))
```

Перевірити що функція isPillDueToday знаходиться у src/features/meds/schedule.js — якщо ні, знайти реальне розміщення командою:

```
grep -r 'isPillDueToday' src/ --include='*.js' -l
```

А.3 Критерій завершення

Задача А вважається виконаною коли:

- npm test показує мінімум 39 тестів (28 існуючих + 11 нових)
- Усі 11 сценаріїв з таблиці вище покриті
- Жоден з існуючих 28 тестів не зламався
- Функція не крашить на null/undefined/порожніх значеннях

ЗАДАЧА Б — CI/CD через GitHub Actions

Налаштувати автоматичний запуск тестів та білду на кожен git push. Це гарантує що жоден коміт не зламає тести або збірку. Особливо критично для медичного застосунку — помилка в prod неприпустима.

Б.1 Файл workflow

Створити файл .github/workflows/ci.yml з таким вмістом:

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test-and-build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: HealthPro-Moie-Zdorovia

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: HealthPro-Moie-Zdorovia/package-lock.json

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build
```

Б.2 Перевірка перед комітом

Переконайтесь що у package.json є скрипт test:

```
grep "test" HealthPro-Moie-Zdorovia/package.json
```

Має бути:

```
"test": "vitest run"
```

Якщо відсутній — додати до scripts у package.json.

Б.3 Push та верифікація

1

Закомітити workflow

- git add .github/workflows/ci.yml
- git commit -m "ci: add GitHub Actions test and build workflow"
- git push origin main

2

Перевірити результат на GitHub

- Відкрити github.com/chironkh-dev/HealthPro-Moie-Zdorovia
- Вкладка Actions → побачити запущений workflow
- Усі кроки мають бути зеленими ☐

Критерій завершення

- GitHub Actions показує зелений статус на останньому коміті
- Значок статусу можна додати до README.md
- Кожен наступний push автоматично запускає тести

РЕКОМЕНДОВАНА ПОСЛІДОВНІСТЬ ВИКОНАННЯ

Крок	Дія	Час	Пріоритет
A.1	Знайти isPillDueToday у коді	5 хв	Спочатку
A.2	Написати 11 тестів у pill-schedule.test.js	1-2 год	Спочатку
A.3	Переконались: npm test → 39+ тестів	5 хв	Спочатку
Б.1	Створити .github/workflows/ci.yml	15 хв	Потім
Б.2	Перевірити скрипт test у package.json	5 хв	Потім
Б.3	git push, перевірити зелений статус Actions	10 хв	Потім
Б.4	Додати badge статусу до README.md	5 хв	Опційно
→	Фаза 2: prx cap add android	Фаза 2	Далі

ВАЖЛИВІ ЗАСТЕРЕЖЕННЯ

НЕ чіпати існуючі 28 тестів

Усі наявні тести `bp-norm`, `bmi`, `health-score` мають залишитись зеленими. Якщо щось зламалось після додавання нових тестів — виправити до коміту.

Не змінювати логіку `isPillDueToday`

Задача — тільки написати тести для існуючої функції, не рефакторити її. Якщо знайдено баг — зафіксувати у коміті окремо від тестів.

CI workflow — тільки `test` та `build`

Не додавати `deploy` у цей workflow. Деплой на GitHub Pages вже налаштовано окремо у `static.yml`. Цей workflow — виключно для перевірки якості коду.

SQLite та Capacitor — після цих двох задач

Не починати Фазу 2 (нpx cap add android) та Етап 5 (SQLite) до зеленого статусу CI. Це страхівка від регресій при нативній збірці.